

CSCI 3308 Project: Verdant

Felix Conant (@ feixxx-jellyfish), Davion Hochhalter (@visorlights), Ian Kyle (@IanShepKyle) ,
Mathias Teferra (@Mathias1100), Maya Williams (@MayaWill)

Project Description

Our application is designed for plant enthusiasts who want a convenient and organized way to document the plants they encounter in nature or cultivate themselves. Users will be able to log individual plant findings by uploading essential information such as names, descriptions, and photos. Each entry will also include the geographic location where the plant was found, allowing users to build a personal, interactive map of their discoveries. This mapping feature adds both functionality and visual appeal, helping users recall exactly where each plant was located and observe patterns in their findings over time.

All uploaded plant data will be stored in a structured database. Once data is collected, users can easily search, filter, and sort through their logged plants using keywords, plant characteristics, or location-based criteria.

This application will be especially valuable for hobby botanists, students, hikers, gardeners, and anyone who enjoys cataloging their natural surroundings. By offering an intuitive interface and interactive visual tools, it makes plant tracking approachable for beginners while still supporting more detailed documentation for advanced users.

Project Tracker

Link to project board: <https://github.com/orgs/CU-CSCI3308-Fall2025/projects/24>

Video: https://drive.google.com/file/d/1ed04b7Q5kEPC0BpQ_7C_TVDLldzkJ9UE/view?usp=sharing

VCS:

Link to repository: <https://github.com/CU-CSCI3308-Fall2025/group-project-feixxx-jellyfish>

Deployment Link: <https://verdant-aamx.onrender.com/>

Contributions

Maya Williams:

- Built our logging plants page which enables users to input information about the plant to add to the database. Created the route in our index.js to direct to this page.
- Testing for registration, checking status returned
- Helped fix database bugs and testing

Ian Kyle

- Built the homepage and stylesheet for the rest of the website. Making the website look somewhat professional and thematically appropriate
- Created the viewPlants endpoint, letting people see the plants that they added or search for.
- Created the search bar feature, allowing users to search for plants and find other related ones. Displays the information and image used for the plant.
- Created the skeleton for the index.js page

Felix Conant

- Built the map page which allows users to view locations of their plant logs and the public plant logs made by other people.
- Helped build the database and adjust the schema to work for the site.
- Connected the map to the database using the /api/plants endpoint.

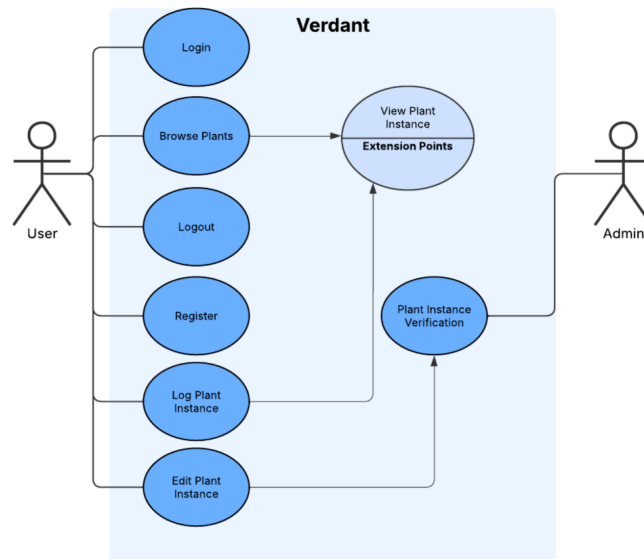
Davion Hochhalter

- Initial database design
- Research (for useful properties of plants to distinguish by)
- Html outline for logging plants page

Mathias Teferra

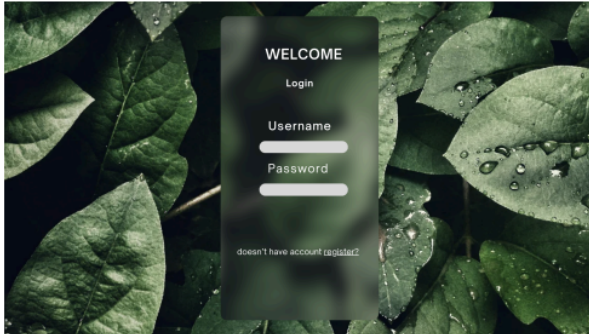
- Created login and registration page with routes in index.js
- Ensure valid password and email
- Changed requirements from username to email for user verification
- Used sendgrid API to send temporary code to current email and allow change if so.

Use Case Diagram

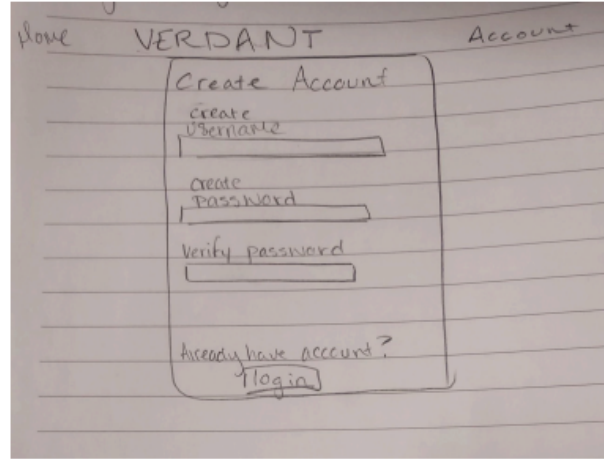


Wireframes (From Lab 8, initial planning)

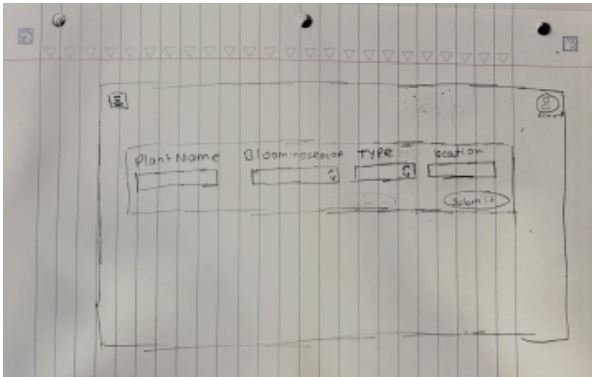
Login



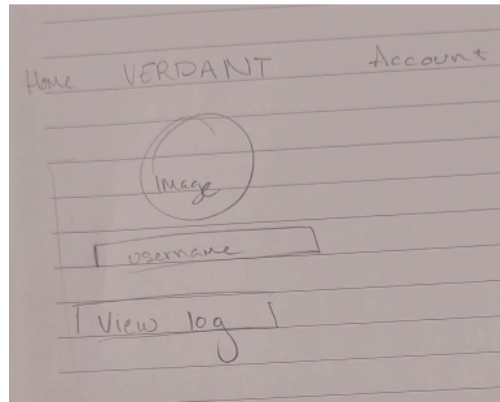
Register



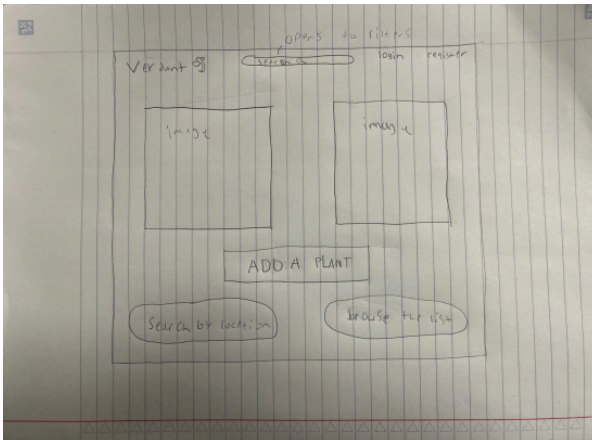
Search



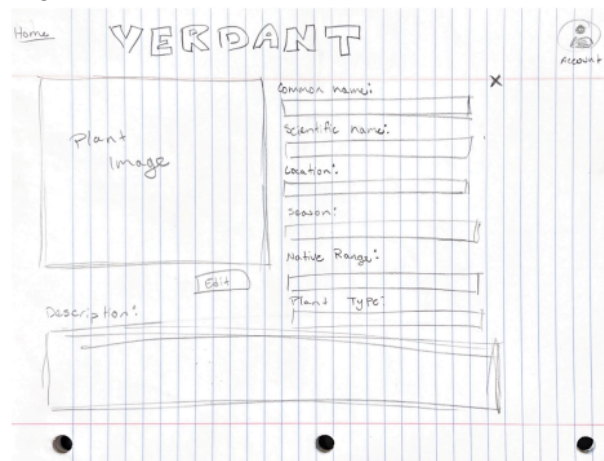
Profile



Home



Log/Edit Plant



Test Results

User testing was done following the test plan which we developed in Lab 10. Four primary features of the application we tested were (1) Plant Logging, (2) Profile Page Display, (3) User Registration, and (4) Map Display of Plant Logs

Testing environments included:

- Localhost development server
- Cloud deployment (Render)
- Chrome and Safari browsers
- Remote PostgreSQL database

Feature one: Logging Plants

Users can log new plants with fields including name, location, scientific name, description, season type, and an optional image URL. Name is required.

Test data: (is it logging correctly?)

Field	Data
Name	Daisy
Scientific Name	Bellis perennis
Location	[40,-105]
Description	Small white flower in backyard
Image url	https://upload.wikimedia.org/wikipedia/commons/thumb/2/27/Bellis_perennis_001.JPG/500px-Bellis_perennis_001.JPG
Season	Spring
Type	Flower

Testing Outcomes/observations:

Use case: The user wants to document plants they find by entering plant details and saving them.

User behavior:

- Users are filling out the fields, however, not all are all filled out unless it is mandatory (plant names)
- More experienced users fill out as much information as possible

User reasoning:

- Users may not know the answer while they log plants,
- May not feel it is necessary to fill out unrequired fields

- Experienced users already have a good background in this topic, therefore they want their tracking details to be as complete as possible

Is their behavior consistent with use cases?

- Yes, users are filling out the logs and feel it is easy to do so
- Some users are not filling them out completely, possibly deviating from our idea of having as much information for logs as possible
- Logs are being saved to the account, allowing users to consistently track plants they find

Why is the behavior deviating?

- Some users entered incomplete information or unexpected formats.

Changes made:

- Having required name to identify the plant
- Clarifying form labels so users know what is expected as input

Feature two: Profile Page with 10 Most Recent Logs

Display email and recent activity (limited to 10 most recent logs). Each log shows name, location, description, and image if given.

Test data: use 15 previously created plant logs for one test user to verify sorting and limit.

Test ID	Scenario	Steps	Expected Result	Actual Result
PF-01	Display recent logs correctly	1. Log in as user with 15 logs. 2. Navigate to profile.	Only 10 most recent logs displayed.	Pass — Limited to 10 entries.
PF-02	Verify sorting order	1. Compare timestamps of displayed logs.	Logs are shown from newest to oldest.	Pass — Sorted correctly.
PF-03	Empty log state	1. Log in as a new user with 0 logs.	Message “No logs yet” displayed.	Pass — Message appears correctly.

Observations:

Use case: Users are navigating to profile pages, checking account information and activity.

User behavior:

- Users are viewing plant information shown from previously logged entries
- Users are ensuring email is correct

User Reasoning:

- They want to verify account information is correct
- They want to see a history of their logged plants
- They want to view information about their plants, rather than just the name

Is their behavior consistent with the use case?

- Yes, users interacted with this consistently, checking logs and information within them

Reason for deviation:

- In our tests, we rarely saw deviation, however some users may want to see more than 10 logs which would require another page of logged activity to render

Changes made:

- We did not do this in the scope of the project, however for future enhancements, our group discussed creating another page for logging activity if the user chooses to see more logs.
-

Feature three: User Registration

Users are able to create an account, emails must be unique and passwords are hashed.

Test ID	Scenario	Steps	Expected Result	Actual Result
UR-01	Register new user	1. Open registration page. 2. Enter valid username, email, password. 3. Submit.	Account created; redirected to login page.	Pass — User successfully registered.
UR-02	Duplicate email check	1. Try to register with same email.	Error: “Email already in use.”	Pass — Error displayed properly.
UR-03	Invalid password format	1. Enter short password (e.g., “abc”).	Error: “Password must be at least 8 characters.”	Pass — Validation enforced.
UR-04	Missing required field	1. Leave “email” blank.	Error message displayed.	Pass — Field validation works.

Observations/ User Outcomes:

Use case: user is creating an account, they are able to log in, and access activities, password chosen meets requirements

Users Behavior:

- Users are creating an account with chosen email
- Some users may be using a simple password
- After registration, they are exploring options allowed after they have logged in

User Reasoning:

- Users may choose a simpler password for easier login, however this may cause security issues
- Using a new email ensures the user has a personal and identifiable account

Is behavior consistent with use cases?

- Yes, registration is familiar for most users as this is commonplace for many applications

- Some users may test different variations for registering to see how strict validation is

Reason for deviation:

- Some users prefer more strict validation to have a safer account, while others care less and want easier log in.
- In testing, users intentionally broke rules (duplicate emails, short passwords) to ensure system responded correctly, though this is expected during testing and not an unexpected deviation

Changes made:

- Password requirements implemented, requiring symbols, numbers and letters and minimum length
-

Feature 4: Map Display of Plant Logs

Logged plants are displayed on an interactive map. Plant entries appear as a marker at its recorded co

Test ID	Scenario	Steps	Expected Result	Actual Result
Map-01	Map loads with default location	Open map page without geolocation	Map centers on [39.7392, -104.9903]	Pass
Map-02	Map loads with user location	Use browser geolocation	Map centers on user location	Pass
Map-03	Current user plant markers	Login as user with plant logs, open map	Plants appear on map, red is private, green is public	Pass
Map-04	Other users public plants	Open map	Other user's public plants are visible, marked as blue	Pass

Observations and User Outcomes:

Use case: User want to view own plant logs and ensure location inputted is correct

- User wants to view public logs

User behavior:

- Users are opening the map page and ensuring geolocation is enabled
- Users are choosing plants to be public or private, and seeing if this appears on the map correctly
- Many interact with the map by zooming and checking location of logs
- Users click on marker to view plant information

User reasoning:

- They want to confirm location was inputted correctly well as the map reflecting this
- Confirming the log was public or private depending on preference

Is behavior consistent?

- Yes, the map does show correct markers and logging information which is what users were clarifying when interacting with the map

Reason for deviation:

- Users may have inputted incorrect coordinates, so plant log is seen at the wrong location

Changes made:

- Also for the future scope of the project, we would like to have the ability for users to view geolocation while they log the plant, to ensure it is correct
- Also, allowing users to input geolocation data, rather than manually inserting coordinates, as this would be more user friendly

ReadMe

```
# Verdant
```

```
## Description
```

```
Our application is designed for plant enthusiasts who want a convenient and organized way to document the plants they encounter in nature or cultivate themselves. Users will be able to log individual plant findings by uploading essential information such as names, descriptions, and photos. Each entry will also include the geographic location where the plant was found, allowing users to build a personal, interactive map of their discoveries. This mapping feature adds both functionality and visual appeal, helping users recall exactly where each plant was located and observe patterns in their findings over time.
```

```
## Contributors
```

```
- Felix Conant  
- Davion Hochhalter  
- Ian Kyle  
- Mathias Teferra  
- Maya Williams
```

```
## Technology Stack
```

- Frontend: HTML, CSS, JavaScript, leaflet.js
- Backend: Node.js
- Database: PostgreSQL
- Version Control: Github repository
- Testing: Mocha, Chai
- Containerization: Docker

Prerequisites

In order to use Verdant, these will need to be installed:

- Node.js
- PostgreSQL
- Docker and Docker Compose

Instructions

****Note:**** when running this application locally, Docker is recommended.

Running tests locally (without Docker)

- Clone the repository and navigate to the project directory
- Install dependencies (npm install)
- Ensure PostgreSQL is running and required environment variables are set.
- Start the application (npm start)
- Visit the application at <http://localhost:3000>

Running locally with Docker (reccomended)

- Start all services using Docker (docker-compose up --build)
- Visit the application at <http://localhost:3000>

Running the tests

This project uses ****Mocha**** and ****Chai**** for backend testing.

Running tests locally (without Docker)

1. Install dependencies (npm install)
2. Run the test suite (npm test)

Running tests with Docker

The ``web`` service in ``docker-compose.yml`` is configured to run ``npm start``, so to run tests inside the container you need to override the command: `docker-compose run --rm web npm test`

This will:

- Build the ``web`` service container if needed
- Mount your project source
- Use the same environment variables
- Run Mocha inside the container
- Exit when done

Tests

- register API: tests `/register` route can successfully register new user, and prevents registering an existing user
- login API: tests `/login` by checking ability to log in with a valid user + password combination, and that it does not allow logging in with a nonvalid user + password combination.

Link to deployed app

<https://verdant-aamx.onrender.com/>